

單元4. 容器資源配置

授課講師:Kevin

目錄：

- .dockerignore建立
- 為什麼要限制資源？
- 容器記憶體設定--memory參數解析
- 讓容器的記憶體爆炸
- docker stats
- docker update
- docker run -restart=always
- docker logs

課程導入

- ▶ 減少image的容量
- ▶ 學會限制容器的 CPU 與記憶體使用量
 - ▶ 透過實際操作，學員將能掌握如何在 Docker 中使用資源限制，進而提高應用的穩定性與效能。
- ▶ 瞭解資源限制對效能的影響
 - ▶ 學員將能分析在限制資源後容器的效能變化，瞭解在不同的配置下應用的運作情形。
- ▶ 實際測試容器在不同限制下的行為
 - ▶ 透過實作，學員將體會不同資源限制對容器行為的影響，並學會如何調整配置以達到最佳效能。

.dockerignore 的基本概念

▶ 定義與功能：

- ▶ .dockerignore 文件的主要作用是告訴 Docker 在執行 docker build 時，哪些本地檔案或資料夾應該被忽略。這一行為與 .gitignore 文件的目的相似，都是為了減少不必要的檔案納入版本控制或建置過程中。

▶ 優化建置過程：

- ▶ 當構建上下文(build context)越小時，Docker 在傳輸檔案的過程中，所需的時間和資源自然會減少，這會直接影響到鏡像的建構速度和最終的鏡像大小。

▶ 避免敏感資料洩露：

- ▶ 使用 .dockerignore 文件可以有效防止敏感資料(如密鑰、密碼、配置檔案等)誤被納入建構的映像中，從而提高安全性。

▶ 使用注意事項：

- ▶ 使用 .dockerignore 時需謹慎，確保不會錯過任何必要的檔案，特別是在大型項目中，可能有許多檔案需要被排除。

.dockerignore 的基本概念

實際操作示範

- ▶ **創建 .dockerignore 文件：**
 - ▶ 在專案的根目錄下 (學員上次建立的 Dockerfile),
 - ▶ 或是下載 <https://github.com/coolyuoo/ClassHello>
 - ▶ 建立一個名為 .dockerignore 的新文件, 並根據實際需求添加要忽略的檔案和資料夾。
- ▶ **建構鏡像：**
 - ▶ 完成 .dockerignore 文件的編輯後, 可以使用 `docker build --no-cache -t <your_image_name>` 命令來構建新的 Docker 鏡像, 您將會發現建構速度有明顯的提升。
- ▶ **效果驗證：**
 - ▶ 在實際建構後, 可以檢查生成的鏡像大小, 並與未使用 .dockerignore 前的鏡像進行比較, 以評估優化效果。
 - ▶ 「.dockerignore 就是減肥清單」—把不需要進 image 的東西瘦下來。
- ▶ **範例內容：**
 - ▶ 任何列出的檔案或資料夾在建構過程中都不會被包含, 這樣可幫助減少最終鏡像的大小。
 - ▶ 繼承上一次的 Dockerfile
 - ▶ 例如, 以下是 .dockerignore 文件的一個示範內容：

名稱	修改日期	類型	大小
.git	2025/10/9 下午 02:51	檔案資料夾	
.idea	2025/10/10 上午 10:50	檔案資料夾	
.venv	2025/10/4 下午 07:27	檔案資料夾	
__pycache__	2025/10/6 上午 09:26	檔案資料夾	
.dockerignore	2025/10/7 下午 07:02	DOCKERIGNORE...	1

.dockerignore 的基本概念

文件格式與範例

__pycache__/

*.pyc

venv/

ENV/

.env

.env.*

```
檔案 編輯 檢視

__pycache__/
*.pyc
venv/
ENV/
.env
.env.*

# Docker build cache
**/.dockerignore
**/Dockerfile
```

.dockerignore 的基本概念

文件格式與範例(2)

基礎 Python／環境與機密

__pycache__/

Python 會自動產生的編譯快取資料夾，對建置映像沒用，只會佔空間。

***.pyc**

Python 編譯後的單一檔案快取(.py 的機器碼快取)，同樣不需要打包進 image。

venv/、ENV/

本機虛擬環境資料夾(pip 安裝在你電腦上的那套)，image 應該在 build 時由 pip install 重新安裝乾淨的一份，不要把本機的 venv 打包進去。

.env、.env.*

環境變數檔(例如 .env.development、.env.production)，常含密碼／金鑰。務必忽略，避免被打包進 image。

.dockerignore 的基本概念

文件格式與範例(3)

「在 .dockerignore 裡, * 就是萬用牌, 代表任何字;

** 是加強版, 代表任何層資料夾;

/ 表示這是個資料夾;

是註解;

! 是反轉, 不要忽略它;

? 是單一字元。

所以 .env.* 就是『任何以 .env. 開頭的檔案都忽略』。」

.dockerignore 的基本概念

總結與反思

▶ 重要性重申：

- ▶ dockerignore 文件在 Docker 生態系統中扮演著重要的角色，它不僅能提高建構效率，還能確保安全性。

▶ 持續學習：

- ▶ 對於開發者而言，熟悉 .dockerignore 的使用及其最佳實踐是非常重要的，這將有助於在實際開發環境中應用 Docker 時，減少不必要的麻煩。

▶ 最佳實踐的探索：

- ▶ 繼續探索其他與 Docker 相關的最佳實踐，並不斷優化工作流程，以便在日常開發中提高生產力和安全性。

休息一下

- ▶ 學員提問

為什麼要限制資源？

- ▶ 避免單一容器耗盡系統資源
 - ▶ 在多容器環境中，若有某一容器資源使用過高，可能影響整個系統的穩定性，因此限制資源是必要的防範措施。
- ▶ 提高多容器環境的穩定性
 - ▶ 透過資源限制，可以確保每個容器都有充足的資源運行，避免因資源競爭導致的服務中斷。
- ▶ 在測試與生產環境控制效能
 - ▶ 限制資源不僅適用於生產環境，測試環境中也需要思考資源的配置，以模擬實際運行情況，發現潛在的問題。

容器記憶體設定--memory參數解析(1)

- ▶ memory是什麼？
 - ▶ 簡單來講, --memory 參數用來限制容器使用的最大記憶體大小, 確保容器不會超過這個限制。
- ▶ 用生活例子比喻
 - ▶ 想像一個人在使用電腦, 若設定了256MB 的記憶體限制, 超出後就會強制關閉過多的應用程式, 這就是 Linux 系統中的 OOM(Out Of Memory)機制。
- ▶ 實際例子
 - ▶ 執行 `docker run -d --name test --memory=256m nginx`, 這個命令將啟動一個只能使用 256MB 記憶體的 Nginx 容器, 若超過限制, 容器將被系統強制停止。

容器記憶體設定-memory參數解析(2)

- ▶ 容器預設可以用主機的全部記憶體。
換句話說，**一個容器就可能吃爆整台主機**。
所以在實際上線環境，一定要記得加上 **--memory** 來保護主機
用生活例子比喻
- ▶ **--memory** 是用來保護主機的安全閥，
限制容器最多可以用多少記憶體，
超過就會被系統強制結束（OOM）。

讓容器的記憶體爆炸(1)

- ▶ 容器內部程式強制結束
 - ▶ 當容器記憶體使用量超過設定的上限時, Linux 的 OOM Killer 將會介入, 強制終止該容器內的主要程式。
- ▶ 服務中斷的影響
 - ▶ 比如, Flask 或 Nginx 等服務將無法運行, 應用程式可能會突然消失, 且 Docker 日誌中通常不會有明顯的錯誤訊息。
- ▶ 主機的安全性
 - ▶ 雖然容器被強制終止, 但這樣的設計能有效保護主機不被單一容器耗盡資源, 避免整體系統崩潰。

讓容器的記憶體爆炸(2)

怎麼知道真的 OOM？

- ▶ 使用 `docker ps -a` 檢查狀態
 - ▶ 可以查看容器的退出代碼，如果是 137，則表示容器因 OOM 被強制終止。
- ▶ 生活化比喻
 - ▶ 就好比一個人在使用筆電時開啟過多程式，結果系統選擇強制關閉最佔資源的程式，以保護整體系統不崩潰。

讓容器的記憶體爆炸(3)

開始模擬容器爆炸

▶ 步驟1: 啟動容器

- ▶ 下載<https://hub.docker.com/r/vsn80395/memoryconfig>
- ▶ `docker run -d -p 80xx:8000 --memory=200m --name yourcontainername vsn80395/memoryconfig:1.0.0`
- ▶ 故意設定較小的記憶體限制, 以模擬記憶體超載情況。

▶ 步驟2: 設定網頁

- ▶ 學生需在啟動後的網頁中輸入記憶體使用量, 進一步觀察容器行為。
- ▶ 使用範例
`/mem/add?mb=150` 加記憶體150mb
`/mem/free?mb=200` 釋放記憶體200mb
`/mem/clear` 記憶體歸0

讓容器的記憶體爆炸(4)

開始模擬容器爆炸

```
localhost:8086  
美化排版 ✓  
{"allocated_mb": 0,  
  "groups": 0,  
  "tip": "搭配 docker stats 觀察容器 RSS/使用率"  
}
```

```
localhost:8086/mem/add?mb=300  
美化排版 □  
{"ok":true,"added_mb":300,"chunk_mb":8,"total_mb":300}
```

```
localhost:8086/mem/free?mb=200  
美化排版 □  
{"ok":true,"freed_request_mb":200,"total_mb":96}
```

讓容器的記憶體爆炸(5)

開始模擬容器爆炸

- ▶ 步驟3: 容器停止
 - ▶ 隨著記憶體超過設定限制200m, 容器將自動停止, 學生可觀察到這一過程。
- ▶ 步驟4: 輸入 `docker ps -a` 查詢開容器狀態 確認容器停止
- ▶ 步驟5: 重啟容器(可多玩幾次)

docker stats

- ▶ 使用 docker stats 命令
 - ▶ 此命令可即時查看所有容器的資源使用情況，包括 CPU、記憶體、I/O 使用狀況，對於管理容器非常有幫助。
- ▶ 實時監控
 - ▶ 學員可以看到每個容器的資源使用百分比，這有助於及時發現性能瓶頸或資源浪費的問題。
- ▶ 分析與調整
 - ▶ 透過這些數據，學員可進行必要的調整，以優化容器的資源配置，提升整體系統效能。
- ▶ 監控輸出示範
 - ▶ 當執行該命令後，會顯示容器名稱及其記憶體用量，如 `NAME MEM USAGE / LIMIT test 130MiB / 256MiB`，這樣能清楚看到容器的記憶體使用狀況。
- ▶ 設定的重要性
 - ▶ 若沒有設置 `--memory`，容器會預設使用主機的全部記憶體，這樣可能造成主機資源耗盡，影響整體系統運行。

docker stats

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
706c6437582a	mmmmmm	0.15%	141.3MiB / 200MiB	70.67%	4.34kB / 1.98kB	0B / 0B	6

CONTAINER ID 容器的唯一識別碼

NAME 容器名稱

CPU % CPU 使用率比例。例：`0.15%`
表示這個容器目前用到整體 CPU 的 0.15%

MEM USAGE / LIMIT 顯示「目前記憶體使用量」與「容器限制 值」。
例：`141.3MiB / 200MiB` →
使用 141.3 MiB, 最大可用 200 MiB

MEM % 目前記憶體使用量相對於限制 值的百分比。

NET I/O 容器的網路收發資料量：`接收 / 傳送`。
例：`4.34kB / 1.98kB` 代表收了 4.34 kB、送出 1.98 kB

BLOCK I/O 容器對儲存裝置的「區塊層 I/O」存取量：
`讀取 / 寫入`。
例：`0B / 0B` 表示目前幾乎沒有磁碟讀寫

PIDS 容器內目前的程序(process)數量。例：`6` 表示這個容器裡有 6 個進程

docker stats

再執行一次記憶體容器，並且另外開一個cmd輸入docker stats觀察容器狀態

docker update

- ▶ 容器運行時可即時修改資源設定
- ▶ 使用 `docker update` 命令，學員能夠在容器運行時動態調整 CPU 和記憶體的配置，這對於處理突發流量或資源需求非常有用。
- ▶ 指令示範
 - ▶ 例如，執行 `docker update 《容器名稱》 --memory=500m`，直接修改容器的記憶體上限，以適應新的需求。
 - ▶ `docker update 706c6437582a --memory=500m --memory-swap=500m`
 - ▶ 解釋一下需要加上 `--memory-swap`

docker update

- ▶ docker update 把 --memory 往上調, 一旦新值 大於目前的 memory-swap, 就會被擋下——這時必須同時帶上 --memory-swap, 讓「 $\text{memory} \leq \text{memory-swap}$ 」成立
- ▶ 調高 memory → 請同時帶 --memory-swap。
- ▶ 調低 memory → 通常不用帶(因為還是 $\leq$ 既有的 memory-swap)。
- ▶ 不想用 swap(教學最乾淨): 把兩個設成一樣。

docker update

- ▶ 使用 docker inspect
 - ▶ 透過 `docker inspect 《容器名稱》 --format '{{.HostConfig.MemorySwap}}'` 來檢查容器的記憶體與交換空間設定, 以便進一步調整。

--memory-swap 是什麼？

- ▶ 理解 Swap Space(交換空間)
 - ▶ Swap 是操作系統在硬碟上劃出的一段空間，當RAM 不夠時，系統會將不常用的資料移到此區域，這樣可以釋放實體記憶體來供需要的程序使用。
- ▶ Swap 的限制
 - ▶ 在 Docker 中，--memory-swap 定義了記憶體和 Swap 空間的總上限，若設定相同則禁用 swap。
- ▶ 效能影響
 - ▶ 雖然 swap 能增加可用記憶體，但由於其速度較慢，使用swap 會影響容器的性能，因此應謹慎使用。

--memory-swap 是什麼？

- ▶ 你可以想成「RAM 是主記憶體」,「Swap 是暫時借硬碟空間來當記憶體」。
- ▶ 類比 意思
- ▶ `--memory` 給容器的「實體 RAM 上限」。
- ▶ `--memory-swap` 「RAM + Swap」的總上限。
- ▶ `--memory-swap - --memory` 就是容器能「借用硬碟空間」的量。

docker run --restart=always

- ▶ 只要容器停止後, 如果再一開始docker run有預先加上這個參數就會每次自動啟動
- ▶ 請學生實際演練, 將容器加上這個參數, 接著測試記憶體爆炸的步驟

休息一下

- ▶ 學員提問

docker logs

- ▶ 想像你有一個日記本,記錄著你每天做了什麼事。Docker logs 就像是容器的日記本
- ▶ **為什麼需要它?** 當你的程式出問題時,你可以看看「日記」裡寫了什麼,找出哪裡出錯了。
- ▶ **下載log產生器的image**
- ▶ <https://hub.docker.com/r/vsn80395/logtest>
- ▶ **基本用法:**
docker logs 容器名稱

docker logs(2)

- ▶ **進階用法：**
- ▶ 即時監看最新 100 行，之後持續跟
`docker logs -f --tail 100 myapp`
- ▶ 看最近 10 分鐘內
`docker logs --since 10m myapp`
- ▶ 指定時間區間(10:00 ~ 11:00)
`docker logs --since "2025-10-30T10:00:00+08:00" \`
`--until "2025-10-30T11:00:00+08:00" myapp`
- ▶ 帶時間戳記
`docker logs -t myapp`
- ▶ 只看最後 50 行
`docker logs --tail 50 myapp`
- ▶ 請學生實際演練，各個參數差別
- ▶ 請學生操作memoryconfig 操作記憶體量 並且另外開一個cmd輸入docker logs來觀察

下載位置

- ▶ <https://github.com/coolyuoo/CpuConfig> 容器CPU控制原始檔
- ▶ <https://hub.docker.com/r/vsn80395/cpuconfig> 容器CPU控制IMAGE
- ▶ <https://github.com/coolyuoo/MemoryConfig> 容器記憶體控制原始檔
- ▶ <https://hub.docker.com/r/vsn80395/memoryconfig> 容器記憶體控制 IMAGE
- ▶ <https://hub.docker.com/r/vsn80395/logtest> LOG產生器IMAGE